# On-Prem Code Generation with Experiment Traceability

*A domain-adaptation pipeline for specialised and low-resource programming languages*

## PROBLEM

Cloud code-generation assistants are off-limits for teams working with proprietary, regulated, or IP-sensitive source code that cannot leave company infrastructure. At the same time, general-purpose code models are trained mostly on popular languages (Python, JavaScript, Java) and perform poorly on legacy, niche, or in-house languages—COBOL, Ada, VHDL, or custom domain-specific languages—where training data is scarce. Small teams and students also lack a reproducible, auditable way to fine-tune and evaluate these models: experiments are run ad-hoc, datasets go unversioned, and results cannot be trusted or repeated.

## VISION

A fully self-hosted code-generation stack that runs entirely on-premises—no code or model weights ever leave the local network. A repeatable domain-adaptation pipeline takes an open base model and specialises it for a chosen target language using lightweight fine-tuning, while every dataset version, training run, hyper-parameter sweep, and evaluation metric is logged to Weights & Biases. The result is measurable uplift on the target language plus a transparent, reproducible audit trail from raw corpus to deployed model.

## TECH STACK

- **Fine-tuning:** LoRA & QloRA & RLHF (DPO/GRPO/etc)
- **Traceability:** Weights & Biases — runs, artifacts (dataset & model versioning), sweeps, tables, model registry
- **Serving (on-prem):** vLLM or Ollama on a local GPU node, exposed through a private API
- **Code generation tool:** Claude Code, OpenCode, etc.
- **Data pipeline:** language-specific corpus collection, de-duplication, cleaning, tokenisation, such as Nvidia Nemo Curator
- **Infrastructure:** Python, Docker, 1-2 GPU (up to 80gb), Git for versioning

## DELIVERABLES

- **Deployment:** working on-prem inference service running an adapted code model
- **Pipeline:** reproducible domain-adaptation scripts (data → fine-tune → evaluate → register)
- **W&B dashboards:** tracked experiments, versioned artifacts, sweeps, and a populated model registry
- **Evaluation report:** base vs. adapted model comparison on target-language benchmarks
- **Documentation & demo:** final report, setup guide, and a live before/after demonstration